Week 6: Trajectory Generation

EE0849 Introduction to Robotics - Detailed Notes

Basri Erdogan

Table of contents

1 Introduction **2**
1.1 Path vs Trajectory . . . 2
1.2 Why Trajectory Planning? . . . 2
1.3 Motion Types . . . 2
1.3.1 Point-to-Point (PTP) . . . 2
1.3.2 Continuous Path (CP) . . . 2

2 Polynomial Trajectories **3**
2.1 Linear Interpolation . . . 3
2.2 Cubic Polynomial Trajectory . . . 3
2.2.1 Boundary Conditions . . . 3
2.2.2 Solution . . . 3
2.2.3 Derivatives . . . 3
2.3 Quintic Polynomial Trajectory . . . 4
2.3.1 Boundary Conditions . . . 4
2.3.2 Matrix Formulation . . . 4
2.3.3 Python Implementation . . . 4

3 Velocity Profiles **5**
3.1 Trapezoidal Velocity Profile . . . 5
3.1.1 Parameters . . . 5
3.1.2 Equations . . . 5
3.1.3 Triangular Profile . . . 5
3.2 S-Curve (7-Segment) Profile . . . 5
3.2.1 Seven Segments . . . 6
3.2.2 Benefits . . . 6
3.2.3 Tradeoffs . . . 6

4 Joint Space vs Cartesian Space **6**
4.1 Joint Space Trajectory . . . 6
4.2 Cartesian Space Trajectory . . . 6

5 Multi-Segment Trajectories **7**
5.1 Via Points . . . 7
5.1.1 Requirements at Via Points . . . 7
5.2 Cubic Splines . . . 7
5.2.1 End Conditions . . . 7

6 Using roboticstoolbox **7**
6.1 Joint Space Trajectory . . . 7
6.2 Cartesian Space Trajectory . . . 8

6.3 Trapezoidal Profile 8

7 Practical Considerations **8**
7.1 Multi-Joint Synchronization 8
7.1.1 Slowest Joint (Standard) 8
7.1.2 Equal Time Scaling 8
7.2 Constraint Checking 8
7.3 Time Scaling 9

8 Summary **9**
8.1 Key Concepts 9
8.2 Profile Selection Guide 9
8.3 Common Mistakes 9

1 Introduction

1.1 Path vs Trajectory

A **path** is a purely geometric description of motion—a sequence of positions in space without any timing information. It answers the question "Where does the robot go?"

A **trajectory** is a path with timing information added. It specifies not only where the robot should be, but when it should be there. A trajectory typically includes:

- Position: $q(t)$
- Velocity: $\dot{q}(t)$
- Acceleration: $\ddot{q}(t)$

1.2 Why Trajectory Planning?

Trajectory planning is essential for several reasons:

1. **Smooth motion**: Prevents jerky starts and stops that can damage mechanisms
2. **Predictable timing**: Allows synchronization with other processes
3. **Motor protection**: Respects velocity, acceleration, and torque limits
4. **Safety**: Ensures controllable, predictable behavior

1.3 Motion Types

1.3.1 Point-to-Point (PTP)

In point-to-point motion, only the start and end positions matter. The path between them is not specified and typically follows the fastest route in joint space.

- Fastest motion type
- Path may be curved in Cartesian space
- Used when only the final position matters (e.g., pick and place)

1.3.2 Continuous Path (CP)

In continuous path motion, the robot must follow a specific geometric path, often a straight line or arc.

- Path is precisely controlled
- Slower than PTP
- Used for welding, painting, machining

2 Polynomial Trajectories

2.1 Linear Interpolation

The simplest trajectory connects two points with a straight line:

$$q(t) = q_0 + \frac{q_f - q_0}{t_f} \cdot t$$

Problem: Velocity is discontinuous at $t = 0$ and $t = t_f$:

- Velocity jumps from 0 to $(q_f - q_0)/t_f$ at start
- Velocity jumps from $(q_f - q_0)/t_f$ to 0 at end
- This requires infinite acceleration, which is impossible

2.2 Cubic Polynomial Trajectory

To achieve smooth motion (zero velocity at endpoints), we use a cubic polynomial:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

2.2.1 Boundary Conditions

We have four unknowns (a_0, a_1, a_2, a_3) and need four equations:

1. $q(0) = q_0$ (start position)
2. $q(t_f) = q_f$ (end position)
3. $\dot{q}(0) = 0$ (start at rest)
4. $\dot{q}(t_f) = 0$ (end at rest)

2.2.2 Solution

Solving these equations:

$$q(t) = q_0 + \frac{3(q_f - q_0)}{t_f^2} t^2 - \frac{2(q_f - q_0)}{t_f^3} t^3$$

Or in normalized time $s = t/t_f$:

$$q(s) = q_0 + (q_f - q_0)(3s^2 - 2s^3)$$

2.2.3 Derivatives

Velocity:

$$\dot{q}(t) = \frac{6(q_f - q_0)}{t_f^2} t - \frac{6(q_f - q_0)}{t_f^3} t^2$$

Acceleration:

$$\ddot{q}(t) = \frac{6(q_f - q_0)}{t_f^2} - \frac{12(q_f - q_0)}{t_f^3} t$$

Note: Acceleration is discontinuous at endpoints (jumps from 0 to $6(q_f - q_0)/t_f^2$)

2.3 Quintic Polynomial Trajectory

For smoother motion, we can also constrain acceleration at endpoints:

$$q(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5$$

2.3.1 Boundary Conditions

Six unknowns require six equations:

1. $q(0) = q_0$
2. $q(t_f) = q_f$
3. $\dot{q}(0) = v_0$ (usually 0)
4. $\dot{q}(t_f) = v_f$ (usually 0)
5. $\ddot{q}(0) = a_0$ (usually 0)
6. $\ddot{q}(t_f) = a_f$ (usually 0)

2.3.2 Matrix Formulation

The coefficients can be found by solving:

$$M \cdot \mathbf{a} = \mathbf{b}$$

Where:

$$M = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & t_f & t_f^2 & t_f^3 & t_f^4 & t_f^5 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 2t_f & 3t_f^2 & 4t_f^3 & 5t_f^4 \\ 0 & 0 & 2 & 0 & 0 & 0 \\ 0 & 0 & 2 & 6t_f & 12t_f^2 & 20t_f^3 \end{bmatrix}$$

2.3.3 Python Implementation

```python
import numpy as np

def quintic_trajectory(q0, qf, v0, vf, a0, af, tf, dt=0.01):
    """Generate quintic polynomial trajectory."""
    M = np.array([
        [1, 0, 0, 0, 0, 0],
        [1, tf, tf**2, tf**3, tf**4, tf**5],
        [0, 1, 0, 0, 0, 0],
        [0, 1, 2*tf, 3*tf**2, 4*tf**3, 5*tf**4],
        [0, 0, 2, 0, 0, 0],
        [0, 0, 2, 6*tf, 12*tf**2, 20*tf**3]
    ])
    b = np.array([q0, qf, v0, vf, a0, af])
    coeffs = np.linalg.solve(M, b)

    t = np.arange(0, tf + dt, dt)
    q = sum(coeffs[i] * t**i for i in range(6))
    v = sum(i * coeffs[i] * t**(i-1) for i in range(1, 6))
    a = sum(i * (i-1) * coeffs[i] * t**(i-2) for i in range(2, 6))
```

```
    return t, q, v, a
```

3 Velocity Profiles

3.1 Trapezoidal Velocity Profile

The trapezoidal profile is widely used in industrial applications. It consists of three phases:

1. **Acceleration phase**: Constant acceleration from rest
2. **Cruise phase**: Constant velocity at maximum speed
3. **Deceleration phase**: Constant deceleration to rest

3.1.1 Parameters

- v_{max}: Maximum velocity
- a_{max}: Maximum acceleration
- d: Total distance

3.1.2 Equations

Acceleration time:

$$t_a = \frac{v_{max}}{a_{max}}$$

Distance during acceleration:

$$d_a = \frac{1}{2} a_{max} t_a^2 = \frac{v_{max}^2}{2a_{max}}$$

Cruise time:

$$t_c = \frac{d - 2d_a}{v_{max}}$$

Total time:

$$t_f = 2t_a + t_c$$

3.1.3 Triangular Profile

If $d < 2d_a$ (short distance), there's no cruise phase:

$$t_a = \sqrt{\frac{d}{a_{max}}}$$

$$v_{peak} = a_{max} \cdot t_a < v_{max}$$

3.2 S-Curve (7-Segment) Profile

The trapezoidal profile has discontinuous acceleration (infinite jerk), which can excite vibrations. The S-curve profile addresses this by adding jerk-limited transitions.

3.2.1 Seven Segments

1. Increasing acceleration (constant positive jerk)
2. Constant acceleration
3. Decreasing acceleration (constant negative jerk)
4. Constant velocity (cruise)
5. Increasing deceleration (constant negative jerk)
6. Constant deceleration
7. Decreasing deceleration (constant positive jerk)

3.2.2 Benefits

- Bounded jerk reduces vibration
- Smoother motion
- Better for precision applications
- Less wear on mechanisms

3.2.3 Tradeoffs

- More complex to compute
- Longer motion time for same limits
- More parameters to tune

4 Joint Space vs Cartesian Space

4.1 Joint Space Trajectory

Planning in joint space means interpolating directly in joint angles:

$$\mathbf{q}(t) = [\theta_1(t), \theta_2(t), \ldots, \theta_n(t)]^T$$

Advantages:

- Simple interpolation
- No IK needed during motion
- Avoids singularities automatically
- Natural motion for the robot

Disadvantages:

- End-effector path is not predictable
- May take longer curved path
- Not suitable for contact tasks

4.2 Cartesian Space Trajectory

Planning in Cartesian space means interpolating in task coordinates:

$$\mathbf{x}(t) = [x(t), y(t), z(t), \phi(t), \theta(t), \psi(t)]^T$$

Advantages:

- Predictable end-effector path
- Straight lines, arcs possible
- Intuitive for operators
- Required for contact tasks

Disadvantages:

- Requires IK at each time step
- May encounter singularities
- Computationally more expensive

5 Multi-Segment Trajectories

5.1 Via Points

When a trajectory must pass through intermediate points (via points), we connect multiple segments.

5.1.1 Requirements at Via Points

- **C0 continuity**: Position matches (always required)
- **C1 continuity**: Velocity continuous
- **C2 continuity**: Acceleration continuous

5.2 Cubic Splines

Cubic splines connect multiple cubic polynomial segments while maintaining continuity.

For n waypoints, we have $n-1$ segments. At each interior point:

1. Position matches from both sides: $q_i^-(t) = q_i^+(t)$
2. Velocity continuous: $\dot{q}_i^-(t) = \dot{q}_i^+(t)$
3. Acceleration continuous: $\ddot{q}_i^-(t) = \ddot{q}_i^+(t)$

5.2.1 End Conditions

Natural spline: Zero acceleration at endpoints

$$\ddot{q}(0) = \ddot{q}(t_f) = 0$$

Clamped spline: Specify endpoint velocities

$$\dot{q}(0) = v_0, \quad \dot{q}(t_f) = v_f$$

6 Using roboticstoolbox

6.1 Joint Space Trajectory

```
import roboticstoolbox as rtb
import numpy as np

# Define start and end configurations
q0 = np.array([0, 0, 0, 0, 0, 0])
qf = np.array([1, 0.5, -0.5, 0, 0.3, 0])

# Generate trajectory (default: quintic polynomial)
traj = rtb.jtraj(q0, qf, 100)  # 100 time steps

# Access results
print(traj.q)    # Joint positions
print(traj.qd)   # Joint velocities
print(traj.qdd)  # Joint accelerations
```

6.2 Cartesian Space Trajectory

```
from spatialmath import SE3

# Define start and end poses
T0 = SE3.Trans(0.4, 0, 0.3) * SE3.RPY([0, 0, 0])
T1 = SE3.Trans(0.4, 0.2, 0.3) * SE3.RPY([0, np.pi/4, 0])

# Generate Cartesian trajectory
traj = rtb.ctraj(T0, T1, 100)

# traj is a list of SE3 poses
for T in traj:
    print(T.t)  # Translation
    print(T.rpy())  # Orientation as RPY angles
```

6.3 Trapezoidal Profile

```
# Trapezoidal velocity profile
traj = rtb.trapezoidal(0, 1, 100)  # 0 to 1, 100 steps

# With explicit times
traj = rtb.trapezoidal(0, 1, np.linspace(0, 2, 100))
```

7 Practical Considerations

7.1 Multi-Joint Synchronization

When multiple joints move different distances, they would naturally take different times. Common approaches:

7.1.1 Slowest Joint (Standard)

1. Calculate time for each joint based on its limits
2. Use the longest time for all joints
3. Scale other joints' velocities down

7.1.2 Equal Time Scaling

All joints complete motion in the same time, but may use different percentages of their limits.

7.2 Constraint Checking

Always verify constraints are satisfied:

```
def check_constraints(q, qd, qdd, q_lim, v_lim, a_lim):
    """Check if trajectory satisfies all constraints."""
    pos_ok = np.all(q >= q_lim[0]) and np.all(q <= q_lim[1])
    vel_ok = np.all(np.abs(qd) <= v_lim)
    acc_ok = np.all(np.abs(qdd) <= a_lim)
    return pos_ok and vel_ok and acc_ok
```

7.3 Time Scaling

If constraints are violated, scale time:

$$t_{new} = t_{original} \cdot k$$

where $k > 1$ slows down the motion.

8 Summary

8.1 Key Concepts

1. **Trajectory = Path + Timing**: Position, velocity, acceleration over time
2. **Polynomial trajectories**: Cubic for velocity continuity, quintic for acceleration continuity
3. **Velocity profiles**: Trapezoidal for simplicity, S-curve for smoothness
4. **Joint vs Cartesian**: Trade-off between simplicity and path control
5. **Multi-segment**: Splines maintain continuity through via points

8.2 Profile Selection Guide

Situation	Recommended Profile
Simple PTP motion	Cubic polynomial
Smooth PTP motion	Quintic polynomial
Industrial motion with limits	Trapezoidal
High-precision motion	S-curve
Through via points	Cubic spline

8.3 Common Mistakes

1. Not checking constraint violations
2. Using Cartesian trajectories near singularities
3. Ignoring jerk limits (causes vibration)
4. Not synchronizing multi-joint motion
5. Using same trajectory for different motion types